

Capstone Project Preparation – Group 8 Technology Survey Report		Page: 1 of 5
Department: Validation	Author(s): Erick, JP, Khurshed, Lauren, Robert	Doc. #: TS-001 Rev. #: 1

1. Introduction

1.1. Objectives

The objective of this document is to examine the different technologies that will serve as the backbone of the Recall Application. We'll achieve this objective through comparing the pros/cons of a technology while considering other alternatives. Based on these comparisons, we'll decide what technology will be chosen for our project.

1.2. Scope

This report will cover four sections of technology/tools that will be used within the project. These sections will include options for front-end frameworks, back-end frameworks, APIs, and libraries. Within each section, we will compare at a minimum of three different options that appear to be viable for our project.

2. Technology

2.1. Front-End Frameworks

This section will evaluate three options that are being considered for the front-end frameworks for the Recall App. Within this section we will compare the benefits and detriments of react native, flutter, and ionic frameworks.

Front-End Framework(s)	Pros	Cons
React Native [1 2]	Large amount of freedom in making design choices. Better performance than alternatives. Many different libraries to work with.	Uses the language React which group members aren't very familiar with.
Flutter [1 2]	Relatively simple to set up, and uses HTML, CSS, and JavaScript.	It has a nested code style, which can be harder to understand (widgets).
Ionic [1 2]	Uses HTML, CSS, and JavaScript, which group members are already familiar with. It has a large amount of UI components. Relatively quick to build and deploy.	Doesn't offer in-app debugging. Performance is comparatively worse compared to Flutter or React Native.

Capstone Project Preparation – Group 8 Technology Survey Report		Page: 2 of 5
Department: Validation	Author(s): Erick, JP, Khurshed, Lauren, Robert	Doc. #: TS-001 Rev. #: 1

The strongest candidate for the front-end of the Recall app appears to be react native. One of the primary benefits to react native is the access to a vast array of prebuilt libraries to choose from. Having access to a dearth of libraries will overall make development easier. Another benefit of react native is that it has either comparable or better performance than the alternatives. While the Recall App isn't expected to need extremely higher performance, it will make users more likely to use the app if it has faster loading-times.

2.2. Back-End Frameworks w/ Integrated Databases

Back-End Framework(s)	Pros	Cons
Django	Django offers simplicity along with many features out-the-box. It provides an integrated database (sqlite3 by default)	Django isn't one the fastest frameworks, this may cause delay when processing large batches of data.
Ruby on Rails ^[10, 12]	Ruby on rails provides a fast-development environment, while also having a large community backing throughout media.	Ruby on rails may lead to larger problems, slowing down the development process – only if inconsistencies are left unchecked during the early stages of development.
Laravel ^[11]	Laravel offers a great ORM which simplifies the database interaction by providing easy and clear syntax.	Laravel is a lightweight framework, requiring more effort for complicated applications. It also has a smaller community.

After careful consideration, we have chosen Ruby on Rails for backend development. While Django was an option, we opted to explore a new technology. Additionally, Ruby on Rails' rapid development capabilities will enable us to build the project within the given timeframe.

Capstone Project Preparation – Group 8 Technology Survey Report		Page: 3 of 5
Department: Validation	Author(s): Erick, JP, Khurshed, Lauren, Robert	Doc. #: TS-001 Rev. #: 1

2.3. APIs

This section highlights the options for data retrieval from the Food and Drug Administration (FDA). We'll examine 3 different ways that we could source data from the FDA in the following table.

API(s)	Pros	Cons
Food API Endpoint ^[3] (openFDA)	Able to search for specific terms easily and well documented. It has the option to download the full dataset and in addition to API calls – all the data is in public domain.	The openFDA has a limit of only 1000 data points per API call, updated weekly (at worst).
Weekly Enforcement Reports ^[4] (daily updates)	The Weekly Enforcement Reports are updated daily, this allows the user to stay up to date with the latest recalls from the FDA. Furthermore, CSV format is easy to parse.	The Weekly Enforcement Report only offers downloads that include the full dataset (.csv file), which means it will have limited datapoints.
Compliance Dashboard-Recalls ^[5]	CDR offers a comprehensive dataset (including ~95K entries)	CDR only offers downloads that include the full dataset (downloads use the .xlsx extension – or Excel XML)

Out of the three options that we have presented, the strongest option available would be the **Food API Endpoint** by openFDA. The benefit behind this choice is that the application won't have to maintain a database of all the recalled items, which could pose reliability issues. Another benefit of using this API, is that it provides more comprehensive data than some of the other alternatives. The limitations of the API are the fact that it has a limit of 1000 datapoints per query – along with the fact that it's updated on a weekly basis. The datapoint limit shouldn't affect our project since historical recall data isn't relevant to the application, along with the fact that if a need does arise to collect more than 1000 points of data, there is the option to make another query while skipping previously returned datapoints. The API's weekly update intervals isn't ideal, but what has been observed in the API, is that updates are often more frequent than weekly.

Capstone Project Preparation – Group 8 Technology Survey Report		Page: 4 of 5
Department: Validation	Author(s): Erick, JP, Khurshed, Lauren, Robert	Doc. #: TS-001 Rev. #: 1

2.4. Libraries

A core feature of the Food Recall App is scanning product barcodes with a mobile device. This function enables the user to check for food recalls quickly and efficiently, reducing the risk of exposure to unsafe products. We evaluated several barcode scanning libraries that are compatible with React Native. These focus on performance, ease of integration, multiplatform support (i.e. Android, iOS), accuracy, documentation quality, and licensing.

Libraries	Pros	Cons
Google ML Kit Barcode API [6 7]	Google ML Kit Barcode offers high accuracy with support for multiple formats. The library is currently in active development. It offers easy integration via Firebase and includes offline capabilities.	Google ML Kit offers features that often require a Firebase setup, which may lead to further complexity. The API is a slightly larger app size, consuming a lot of storage from the user.
ZXing (Zebra Crossing) [8]	ZXing is a lightweight, open-source library providing support for various barcode formats. It's a great choice for integrating barcode scanning features. It's also compatible with multiple platforms enhancing reliability.	ZXing's integration with React Native would require third-party wrappers – and it'll also be less accurate than Google ML Kit.
Dynamsoft Barcode Reader [9]	Dynamsoft offers extremely fast and accurate readings. It's commercial-grade quality software and has an extensive support for various barcode formats.	The main drawback of Dynamsoft is that a license is required to unlock all features. Additionally, it has limited community support.

After careful consideration, we selected **Google ML Kit Barcode Scanning API** as the barcode scanning technology for the Food Recall App. Offering strong balance between accuracy, offline capabilities, cross-platform support, and ease of integration with React Native via the *react-native-google-ML-kit* package. It supports various barcode formats (i.e. QR Code, UPC, EAN) that are crucial for grocery item recognition.

Capstone Project Preparation – Group 8 Technology Survey Report		Page: 5 of 5
Department: Validation	Author(s): Erick, JP, Khurshed, Lauren, Robert	Doc. #: TS-001 Rev. #: 1

While ZXing is a great option, integration with React Native requires 3rd party libraries adding extra complexity. Additionally, it doesn't match detection speeds as compared to Google ML Kit. Dynamsoft, while highly accurate and enterprise-grade, was ruled out due to its commercial licensing cost which is we believe is out of scope.

3. Sources

- [1] React, Flutter, and Ionic Article <https://www.deduxer.studio/blog/ionic-vs-react-native-vs-flutter-whats-best-for-2024#:~:text=React%20Native%20excels%20in%20projects,most%20powerful%20in%20every%20aspect.>
- [2] Comparison of developing with React Native, Flutter, and Ionic <https://medium.com/@fmmagalhaes/i-built-the-same-app-with-flutter-react-native-and-ionic-33ff8b358562>
- [3] openFDA API <https://open.fda.gov/apis/food/enforcement/>
- [4] FDA Weekly Enforcement Reports https://www.accessdata.fda.gov/scripts/ires/index.cfm#tabNav_advancedSearch
- [5] FDA Compliance Dashboard- Recalls <https://datadashboard.fda.gov/ora/cd/recalls.htm>
- [6] Google ML Kit Barcode Scanning API Documentation. Google Developers. <https://developers.google.com/ml-kit/vision/barcode-scanning>
- [7] react-native-google-mlkit. GitHub. <https://github.com/react-native-ml-kit/barcode-scanning>
- [8] ZXing Project Page. GitHub. <https://github.com/zxing/zxing>
- [9] Dynamsoft Barcode Reader SDK. <https://www.dynamsoft.com/barcode-reader/overview/>
- [10] Ruby on Rails Managing Databases. https://guides.rubyonrails.org/v2.3/getting_started.html#configuring-a-database
- [11] Introduction to Laravel. <https://laravel.com/docs/12.x/database#introduction>
- [12] Advantages & Disadvantages of Ruby. <https://www.bluebash.co/blog/ruby-on-rails-pros-and-cons>